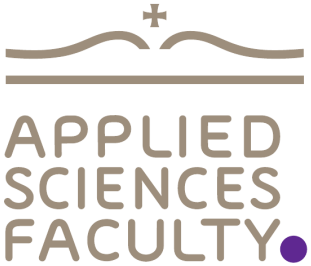
C++ — лекція 11. Класи — 1. Код + дані





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Дані і код – процедурний підхід

Нехай у нас є задача написати бібліотеку, яка дозволятиме працювати із раціональними числами.

- Раціональні числа — нескоротні дробів із цілим чисельником і натуральним знаменником.
- В звичних програмістам термінах, цілі числа — це int, натуральні — unsigned int, тобто – завжди додатні[1].

[1] Однак, для простоти внутрішньої реалізації, ми використовуватимемо і для знаменника int, хоча й вимагатимемо, щоб в кінцевих результатах він був додатним.

Дані і код – процедурний підхід

Дані зберігаємо у структурі[2]:

```
struct SimpleRatio {
    int chys, znam; // numerator, denominator>0
};
```

Код – знаходиться в функціях:

```
SimpleRatio add(SimpleRatio lhs, const SimpleRatio& rhs){
    lhs.chys = lhs.chys * rhs.znam + lhs.znam * rhs.chys;
    lhs.znam *= rhs.znam;
    return reduce(lhs);
}

SimpleRatio& reduce (SimpleRatio *arg){
     .....
    // Приведення до канонічної форми:
    // – Знак в чисельнику, знаменник більший за нуль
    // – Скорочення
    // – 0/N перетворюємо в 0/1
    return *arg;
}
```

Дані і код – процедурний підхід

Використання

```
SimpleRatio res, d={2, 3}, r={5, 7};

reduce(d);
reduce(r);

res = add(d, r);
cout << d << " + " << r << " = "
     << res << endl;

cout << d << " == " << r << " is "
     << isEqual(d, r) << endl;
```

Клас – код плюс дані

```
class SimpleRatio {
public: // Тимчасово
    int chys, znam;

// Тимчасово! На практиці
// робитимемо по іншому!
 SimpleRatio add(SimpleRatio rhs){
  SimpleRatio res;
  res.chys = chys * rhs.znam
           + znam * rhs.chys;
  res.znam = znam * rhs.znam;
  return reduce(res);
 }
 void reduce(){     .....     }
 bool isEqual(SimpleRatio rhs){
  return chys == rhs.chys
      && znam == rhs.znam;
 }
};
```

Використання:

```
SimpleRatio res, d={2, 3}, r={5, 7};

d.reduce();
r.reduce();

res = d.add(r);
cout << d << " + " << r << " = "
     << res << endl;

cout << d << " == " << r << " is "
     << d.isEqual(r) << endl;
```

В реальному коді дії типу додавання варто реалізовувати по іншому, крім того, для лаконічності не використано декілька важливих ідіом, наприклад, передача за константним посиланням. Про конструктори, константні методи, розмежування доступу і т.д. — буде далі.

Класи та об'єкти

- Клас — як і структура, тип даних, що створюється програмістом, та містить дані і методи роботи із цими даними[3].
- Чисто технічно, клас — об'єднання даних і коду, хоча його суть в іншому:
- Концептуально клас є основною одиницею побудови абстракцій.
- Змінна типу класу називається об'єктом.
- Класи та об'єкти є елементарними частками використання концепції об'єктно-орієнтованого програмування (ООП).
- Однак, увага, наявність в програмі ключового слова class чи й повноцінних класів не робить програму чи бібліотеку об'єктно орієнтованою! ООП — ідеологія проектування програмного забезпечення. Підтримка класів мовою програмування лише спрощує її застосування.

---

[3] Зацитую також вікіпедію: *"В об'єктно-орієнтованому програмуванні, клас — це спеціальна конструкція, яка використовується для групування пов'язаних змінних та функцій. При цьому згідно з термінологією ООП глобальні змінні класу (члени-змінні) називаються полями даних (також властивостями або атрибутами), а члени-функції називають методами класу. Створений та ініціалізований екземпляр класу називають об'єктом класу. На основі одного класу, можна створити безліч об'єктів, що відрізнятимуться один від одного своїм станом (значеннями полів)."*

Оголошення та визначення методів

- Функції у класі називаються **методами**.
- Тіло функції-метода може знаходитися як безпосередньо в оголошенні класу, так і окремо[4].

```
class SimpleRatio {
        .....
 SimpleRatio add(const SimpleRatio &rhs);

 inline bool isEqual(SimpleRatio rhs){
  return chys == rhs.chys && znam == rhs.znam;
 }
};

 SimpleRatio SimpleRatio::add(const SimpleRatio &rhs){
       .............
 }
```

- Враховуючи типову організацію програми — клас оголошується в файлах заголовків (header), в тілі рекомендується залишати визначення лише inline-функцій.

[4] Такий інтерфейс додавання використовуємо тимчасово! На практиці робитимемо по іншому!

- Як і незалежні ф-ції, методи можна перевантажувати. Звичайно, вибір перевантаження відбувається в межах класу[5] — методи різних класів можуть називатися так само, абсолютно не впливаючи один на одного.
- Метод може використовувати всі інші члени класу, як поля, так і методи, не залежно від того, в якому порядку вони оголошені. Зокрема, на відміну від областей видимості, можна використовувати і ті елементи класу, що були оголошені нижче.
- Деякі міркування по вибору підходу до створення API: `https://www.foonathan.net/2022/03/behavior-function-type/`.
- "API naming principle: If there is no direct object, then the direct object is the source object" – підказка до вибору правильних назв.

[5] Деталі, пов'язані із успадкуванням, розглядатимуться пізніше.

Конструктори

- Робити кожного разу щось типу:

```
SimpleRatio d;
d.chys = 2;
d.znam = 3;
d.reduce();
```

  незручно. А це ж ще зовсім примітивний приклад.

- В цьому випадку є трішки зручніший спосіб:

```
SimpleRatio d={2, 3};
d.reduce();
```

  Однак і цей метод не знімає проблеми — всі дії по ініціалізації об'єкту виконуються ззовні нього.

- Хоча, можна для ініціалізації написати якусь функцію, типу "`initRation`", але C++ надає спеціалізований засіб — **конструктори**.

:

Конструктор

- Спеціальний **метод**, оголошений в класі, який **має те ж ім'я**, що і клас.
- Викликається під час оголошення змінної типу даного класу. Створює відповідний об'єкт.
- Не повертає ніякого значення.
- Може приймати аргументи.

Конструктор — приклад

```
class SimpleRatio{
        ..................
    SimpleRatio(int c=0, int z=1)
    {
        chys = c;
        znam = z;
        reduce();
    }
};
```

Просте присвоєння можна робити трішки простіше, із використанням списку ініціалізації:

```
    SimpleRatio(int c=0, int z=1):
        chys(c), znam(z)
    {
        reduce();
    }
```

Список ініціалізації

- Код типу згаданого вище, із списком ініціалізації:

```
SimpleRatio(int c=0, int z=1)    : /*<===*/
    chys(c), znam(z)               // <====
{
    reduce();
}
```

  використовує для chys і znam семантику ініціалізації.
- Він є необхідним для об'єктів-членів даних та сутностей, які не можна змінювати — констант, посилань, тощо.
- Також так зручно ініціалізувати об'єкти-члени із своїми конструкторами.
- ◇ Якщо клас поля не має конструктора без аргументів, це єдиний спосіб передати аргументи його конструктору — без них код не компілюватиметься.
- ◇◇ В C++11 можна ініціалізувати під час оголошення, але єдиним для всіх конструкторів способом.
- ◇ Навіть якщо такий конструктор існує, часто неефективно створювати об'єкт за замовчуванням[6], а потім змінювати/перетворювати/заміняти його на потрібний.
- **Увага!** ініціалізація членів таким способом здійснюється в порядку їх оголошення в класі!

[6] Конструктори без аргументів називаються конструкторами за замовчуванням.

Створення об'єкту

```
class SimpleRatio{
        ..................
    SimpleRatio(int c, int z):
        chys(c), znam(z)
    {
        reduce();
    }
};

int main()
{
    SimpleRatio drib (5,3);
}
```

Однак, якщо конструктор є лише такий, як вище на цьому слайді, то наступний код буде некоректним:

```
SimpleRatio drib2;
```

Конструктор за замовчуванням — конструктор без аргументів

- Конструктор за замовчуванням:

```
class SimpleRatio{
    ......................
    SimpleRatio():
        chys(0), znam(1)
    {
    }
};
```

- Викликається, якщо під час створення об'єкту не передано жодних аргументів:

```
SimpleRatio drib2;
```

- Есей на тему: "What to do if you don't want a default constructor?".
- "How can I choose a different C++ constructor at runtime?"

Згенерований компілятором конструктор за замовчуванням

- Якщо в класі не оголошено жодного конструктора, компілятор створює конструктор за замовчуванням.
- Цей автоматично згенерований конструктор:
  - Викликає конструктор базового класу за замовчуванням.
  - Для кожного поля типу класу викликає його конструктор за замовчуванням.
  - Поля фундаментальних (вбудованих) типів не ініціалізуються.
- C++11 дозволяє явно попросити його створити:

```
class C{
        C() = default;
};
```

- Або явно заборонити його створення:

```
class C{
        C() = delete;
};
```

Конструктори фундаментальних типів

- З фундаментальними типами є один нюанс[7].
- Нехай у класу Class є поля фундаментальних типів, але немає створеного програмістом конструктора.
- Тоді, у наступних ситуаціях поля таких типів цього класу:

```
Class obj1;                  // не ініціалізуються
Class obj2 = Class();        // ініціалізуються нулями
Class* pobj3 = new Class;    //не ініціалізуються
Class* pobj4 = new Class(); // ініціалізуються нулями
```

- Додатково, це – не ініціалізація взагалі:

```
Class notobj();      // НЕ СТВОРЕННЯ ОБЄКТУ! Оголошує функцію.
```

- Однак, не варто покладатися на таку поведінку.
- В C++11 є інші методи ініціалізації. Про них пізніше[8].

---

[7] За деталями див. http://en.cppreference.com/w/cpp/language/value_initialization, http://en.cppreference.com/w/cpp/language/default_initialization і http://en.cppreference.com/w/cpp/language/zero_initialization.

[8] Див., зокрема, http://en.cppreference.com/w/cpp/language/list_initialization

Явні виклики конструктори фундаментальних типів

- На відміну від конструкцій з попереднього слайду, ось такий код використовується доволі широко (в основному, в шаблонах):

```
int x = int(); // x == 0
double y = double(); // y == 0.0
```

- Тобто, мова вважає, що для фундаментальних типів теж існують конструктори за замовчуванням – T(), які ініціалізують їх нулем або еквівалентним до нього значенням (наприклад, nullptr).
- Однак, хоча їх можна викликати явно, автоматично вони не викликаються:

```
int x;
std::string y;
```

Хоча y буде ініціалізовано, x – ні.

◇ Зроблено так, поміж іншого, заради сумісності із поведінкою C.

Перевантаження конструкторів

- Конструктори теж можна перевантажувати:

```
class SimpleRatio{
    ......................
    SimpleRatio():
        chys(0), znam(1)
    { }
    SimpleRatio(int c, int z): chys(c), znam(z){
        reduce();
    }
};
```

◇ Звичайно, в коді вище цього можна досягнути одним конструктором із параметрами за замовчуванням:

```
class SimpleRatio{
    ......................
    SimpleRatio(int c=0, int z=1): chys(c), znam(z) { reduce(); }
};
```

◇ Однак, така можливість є далеко не завжди (дроби — дещо специфічний клас, як і інші класи-величини, особливо — "математичні").

Ініціалізація членів класів під час оголошення (C++11)

- C++11 додав можливість ініціалізувати члени даних класу безпосередньо під час оголошення.

```
class SimpleRatio{
    int chys = 0;
    int znam = 1;
    SimpleRatio() = default;
    SimpleRatio(int c, int z): chys(c), znam(z) {
        reduce();
    }
};
```

- Ця ініціалізація використовується, якщо конструктор не ініціалізує їх по іншому.
- Фундаментальні типи можна ініціалізувати під час оголошення за допомогою '=', однак користувацькі типи – лише за допомогою узагальненої ініціалізації.

```
class SimpleRatioTagged{
        .....................
    std :: string  name{"Default"};
};
```

- Поміж іншого можна виділяти пам'ять чи використовувати виклики функцій:

```
class SimpleRatioTagged{
        SomeType *ptr = new SomeType{};
          // А ось 'new SomeType;' -- не можна.
        SomeType def{getDefaultSomeType()};
        int t = getIntForSomeType();
};
```

Тривіальний конструктор за замовчуванням

- Важливим конструктором за замовчуванням є **тривіальний конструктор за замовчуванням**.
- До нього є наступні вимоги:
  - Він не створений користувачем, або явно створений за замовчуванням: `C()=default;`.
  - Клас не має віртуальних функцій.
  - Клас не має віртуальних базових класів.
  - Клас не ініціалізує не-статичні члени при оголошенні.
  - Всі базові класи мають тривіальні конструктори.
  - Всі не-статичні члени мають тривіальні конструктори.
- Це конструктор, який не виконуватиме жодних дій.
- Класи, сумісні із C (POD) мають мати тривіальний конструктор за замовчуванням.

Двофазна ініціалізація

- Іноді ініціалізувати повністю в конструкторі нераціонально або неможливо, з різних причин.
- ◇ Хоча такої ситуації потрібно уникати.
- Тоді може використовуватися двофазна ініціалізація – конструктор здійснює базову підготовку, а виклик методу, наприклад *init(...)*, завершує її.
- ◇ В таких випадках варто турбуватися про дотримання інваріантів класів, принаймні у режимі відлагодження – якщо з міркувань продуктивності це неможливо робити завжди. Проблема тут у загрозі використання не повністю ініціалізованого класу.
- "Mixing N-phase Initialization" – есей на тему.

Протилежна задача — деструктори

- Якщо конструктор виділяє динамічну пам'ять, відкриває файли чи захоплює інші ресурси (мютекси, сокети і т.д.), їх слід звільняти, коли об'єкт припиняє існування.
- Щоб мати нагоду автоматично все звільняти при знищенні об'єкта, використовується спеціальна функція — деструктор. Його ім'я — ім'я класу, перед яким стоїть значок тильда, "~":

```
class TSomeClass{
public:
    int *pi;
    TSomeClass(){
        pi = new int;
    }
    ~TSomeClass(){
        delete pi;
    }
};
```

- Не повертає значень і не приймає аргументів.

Деструктори

- Не повертає значень і не приймає аргументів.
- Викликається автоматично, коли об'єкт виходить із області видимості.
- Також викликається деструктор під час виклику delete для динамічно виділених об'єктів.
  - ◇ Для динамічних масивів він викликається для кожного елемента.
- Якщо деструктор не написано, компілятор створює деструктор за замовчуванням. Він:
  - Викликає деструктори членів даних, які є об'єктами.
  - Є віртуальним, якщо в котрогось із предків був віртуальний деструктор.
- Наявність деструктора дозволяє ефективно обходитися без очищувача пам'яті (garbage collector). При чому, звільнення ресурсів відбувається більш контрольовано.
  - ◇ Хоча й не абсолютно, наприклад, виключення під час розкручування стеку може спричинити лавину викликів деструкторів, ефективно створюючи ситуацію, схожу на невчасне втручання автоматичного очищувача – "зупинку світу".

Деструктори

- Засіб з схожими функціями, запропонований для наступної версії C як TS в 2025 – defer: "The Defer Technical Specification: It Is Time".
- "Forward declaring a type in C++: The good, and the bad" – для попередньо оголошених типів, визначення яких не бачить компілятор на момент виклику деструктора, він вважатиме деструктор тривіальним. Це UB, і є надія на попередження від компілятора.
  - ◇ "C++20s concepts with a forward declared type" – продовження теми.

Константні функції-методи

- Якщо метод не змінює даних класу, варто про це повідомити явно:

```
class SomeClass{
    int getField() const
    {
        return field;
    }
}
```

- Технічно це відповідає тому, що неявно переданий методу вказівник this вказує на константні дані.
- Якщо об'єкт оголошено як const, для нього можна викликати лише константні методи.
- Чисто технічно, вміст константного об'єкта все рівно може змінюватися. Див. далі.

Вказівник **this**

- Методам об'єкту (якогось класу) автоматично доступний спеціальний вказівник на сам цей об'єкт — той, для якого їх було викликано.
- Називається він **this**, і має тип "**`ClassName *`**" для неконстантних методів та "**`const ClassName * `**" для константних. Однак, при тому він вважається rvalue — йому не можна нічого присвоювати (в цьому веде себе схоже на "**`ClassName * const`**"), також не можна взяти його адресу.
- Фактично, this передається методам, як "нульовий", невидимий/неявний аргумент. Точніше, фактично передається не вказівник, а посилання на об'єкт, але методам ця сутність доступна саме як вказівник.

Тобто, звертання до полів:

```
class A{
    int m;
public:
    void fn(){
        m=5; // <==========
    }
};
```

еквівалентне до:

```
this->m=5;
```

Явне використання this майже ніколи не потрібно. (Виняток: звертання до членів базового класу, успадкованого шаблоном.)

- Через те, що "нульовим" аргументом передається посилання на об'єкт, для якого викликається метод, перевизначення оператора, як методу, може бути тотожним його перевизначенню у вигляді незалежної функції.
- Тобто, наступний код дасть помилку, типу: "ambiguous overload for 'operator!' (operand type is 'A')" (gcc-4.8):

```
class A{
public:
    A operator!() const { .... };
};
A operator!(const A& a) { .... };
```

- (Та ж проблема буде і для пари "`A A::operator!()`" і "`A operator!(A& a)`", однак, така форма використовується рідко! Деталі — коли буде мова про перевантаження операторів.)
- При тому, наступні функції вважатимуться різними, компілятор "розбереться":

```
class A{
public:
    void fn(){}
};

void fn( A& a){}
```

Приклад використання this – 1

Перевірка, чи аргументом не передано той же самий об'єкт, чий метод працює:

```
class A
{
    .....
public:
    .....
    bool is_it_me(const A& a)
    {
        return &a == this;
    }
    .....
}
```

Повернення посилання на себе

Якщо є потреба вибудовувати ланцюжки із викликів методів, можна повернути посилання на самих себе:

```
class N{
      .......
    N& add(const N& a)
    {
          ......
        return *this;
    }

    N& mul(const int n);
      .......
}

    N s,k;
     ...
    s.add(k).mul(5).add(s);
```

Перевантаження за константністю та l/r-valueness

- Оскільки неявним аргументом методів є об'єкт, для якого ці методи викликаються, є можливість перевантажувати за їх властивостями.
- Це може бути корисним, наприклад, якщо для одного з варіантів можлива швидша реалізація, або для використання в різних контекстах.
- До C++11 був лише один спосіб[9]:

```
class Test2{
    ....................
    void method()       {   ...........   }
    void method() const {   ...........   }
};
```

Перший буде обрано для не-константних об'єктів, другий – для константних.

- В C++ можна використати альтернативний спосіб[10]:

```
class Test1{
public:
    void method()       &  {   ...........   } // 1
    void method() const &  {   ...........   } // 2
    void method()       && {   ...........   } // 3
    void method() const && {   ...........   } // 4, Зазвичай не дуже корисний
};
```

[9] Аналогічні конструкції можливі для volatile, ми таку екзотику тут не будемо розглядати.

[10] Але тоді – лише такий, змішувати не можна.

- Приклад вибору відповідних методів у різних ситуаціях:

```
class Test1{
public:
    void method()       & { ..... } // 1
    void method() const & { ..... } // 2
    void method()       && { ..... } // 3
    void method() const && { ..... } // 4, Зазвичай не дуже корисний
};

Test1 get_test1();


  .....
    Test1 t1;
    const Test1 t1c;
    t1.method();                // 1
    t1c.method();               // 2
    std::move(t1).method();     // 3
    get_test1().method();       // 3
    std::move(t1c).method();    // 4
```

Статичні члени

- Класи можуть містити статичні поля та методи.
- Статичні дані — єдині для всіх екземплярів даного класу.
- Звертатися до таких полів можна як до звичайних полів: `obj.static_data1`, або не вказуючи конкретного об'єкта, конкретного екземпляра, якого вони стосуються: `TObj::static_data1` — вони єдині для всіх екземплярів.
- Аналогічно, статичні методи — єдині для всіх екземплярів класу. Фактично, це теж глобальні функції, доступні лише в його контексті. Такі функції, не отримуючи this — вони не стосуються конкретного об'єкта, можуть використовувати лише статичні дані.
- Крім всіляких трюків із підрахунком об'єктів, приклади яких далі, статичні члени корисні для створення різноманітних значень за замовчуванням, доступних лише класу.

Статичні дані

- Статичні дані — єдині для всіх екземплярів даного класу:

```
class SomeClass{
      .....
     static int Objects_number;
      .....
     SomeClass () {
           .....
          ++Objects_number;
     };
     ~SomeClass () {
           .....
          --Objects_number;
     };
      .....
};
   ..............
int SomeClass::Objects_number = 0;
```

- Фактично, статичні члени є глобальними змінними, доступними лише в контексті класу, якому вони належать.
- Для того, щоб під ці змінні було виділено пам'ять, їх потрібно ініціалізувати рівно один раз, у котромусь із файлів реалізації (.cpp).
- ◇ В C++17 це також можна зробити за допомогою inline-змінних.

Статичні константи інтегральних типів – 1

- Статичні константи інтегральних типів можна повністю задати в оголошенні класу[11]:

```
class SomeClass{
      .....
     static const int def_code=1111;
      .....
};
```

- Визначати такі члени за межами класу потрібно тоді і тільки тоді, коли буде потреба знати їх адресу:

```
const int SomeClass::def_code;
  .......
const int *de = &SomeClass::def_code;
```

  Вказувати ініціалізатор повторно не можна!

◇ Це досі (C++20) спричиняє загадкові проблеми – коли в одному режимі (з оптимізацією) функція, яка отримує посилання на такий об'єкт, заінлайнена, а в іншому (відлагоджувальному чи з іншими ключами оптимізації) – ні. Тоді в одному режимі код нормально лінкується, а в іншому – лінкування ламається. Приклад: `https://quuxplusone.github.io/blog/2020/09/19/value-or-pitfall/`.

◇ Вирішення – використовувати замість них static constexpr-об'єкти.

[11] Стандарт C++11 дещо пом'якшує строгість цих вимог.

Статичні константи інтегральних типів – 2

- Для створення набору констант в класі зручно може бути використати enum:

```
class SomeClass{
      .....
     enum colors{RED, GREEN, BLUE, DEF_CODE=1111};
      .....
};
```

Статичні inline-змінні

- Починаючи із C++17, завдяки поняттю inline-змінних, визначати безпосередньо в оголошенні класу можна змінні будь-яких релевантних типів:

```
class SomeClass{
    .....
    static const int def_code=1111;
    static inline std::string str1 = "Hello Class";
    static inline std::string str2 {"Hello Class"};
    .....
};
```

- Зауважте, на відміну від ініціалізації членів класу, фігурні дужки – узагальнена ініціалізація, не є обов'язковими.

Статичні методи

- Змінимо та доповнимо попередній клас так:

```
class SomeClass{
private:
    static int Objects_number;
     .....
public:
     ......
    static int get_objects_number();
     ......
};
int SomeClass::get_objects_number()
{
       return Objects_number;
}
 ....
        SomeClass a;
    ...
        cout << SomeClass::get_objects_number() << endl;
        cout << a.get_objects_number() << endl;
```

const-коректність і mutable

- Взагалі, константність — річ корисна, вона радикально спрощує багато що в розробці програм (від дизайну та пошуку помилок, до реалізації багатопоточності).
- Тому варто максимально нею користуватися. Зокрема, всі методи, які не повинні змінювати об'єкт, для якого викликані, слід оголошувати константними.
- Однак, часто виникає конфлікт між двома поглядами на константність:
  - фізичною константністю, коли жоден біт об'єкта не змінюється?
  - логічною константністю, коли незмінним залишається лише логічний стан об'єкта, а фізично він може змінюватися
- Приклад логічної, але не фізичної, константності — класи із підрахунком посилань, для уникнення зайвих копіювань, класи із кешем.
- **Увага:** константність об'єктів не є глибокою. Наприклад, якщо клас містить вказівник на не константні дані, то, навіть для константного об'єкта, ці дані можна змінювати, хоча сам вказівник – вже ні. А ще, константний метод може повернути не константний вказівник чи посилання на внутрішні дані, і тоді...

- Уявимо собі, що в класі дробу є метод toString(), який за першого виклику генерує текстове представлення, і зберігає його, щоб повернути за наступних викликів. Практично, він тут явно надмірний — збільшує розмір об'єкту, але для ілюстрації зійде.
- Це можна реалізувати, наприклад, так[12]:
  - Метод toString() перевіряє, чи немає вже готової стрічки в кеші, якщо є — використовує її, інакше створює і повертає. При тому, відмічає (у булівській змінній), що кеш містить коректну стрічку.
  - Кожен із методів, змінюючи дріб, повинен інвалідувати кеш – за допомогою тієї ж булевої змінної. (Так як більшість таких методів викликатимуть метод reduce(), можна покласти цю відповідальність на нього[13].
- Функція toString(), логічно є константною — вона не змінює об'єкт. Однак, стрічку кешу, та прапорець її валідності, вона повинна могти змінювати. Для цього їх оголошують із специфікатором **mutable**.
- Такий підхід (див. також приклад на наступному слайді) варто застосовувати, коли "мутабельних" полів мало. Якщо їх потрібна помітна кількість, краще створити допоміжний клас, що реалізовуватиме оцю змінну частину константних об'єктів, і в головному класі зберігати вказівник на неї (можливо, хоч і не обов'язково – завдяки неглибокій константності, мутабельний).

[12] Реалізація ретельно не тестувалася! Чисто демонстрація підходу. Крім того, не релевантні елементи класу не показані, їх присутність має бути очевидною з контексту.

[13] Такі кеші вимагають великої акуратності в реалізації! Зокрема, з точки зору багатопоточності.

```
class RatioCached{     .................
    int ch, zn;
    mutable bool cache_is_valid;
    mutable string* string_cache;

    RatioCached(int c=0, int z=1):
        ch(c), zn(z), cache_is_valid(false), string_cache(0)
    {
            reduce();
    }
    void reduce(){
        // Приводимо дріб до канонічного вигляду
        cache_is_valid = false;
    }

    string toString() const{
        if(cache_is_valid)
            return *string_cache;
        else{
            if( !string_cache ) string_cache = new string;
            // .... Генеруємо текстове представлення
            cache_is_valid = true;
            return *string_cache;
        }
    }
    ~RatioCached(){ delete string_cache; }
```

```
class RatioCached2{    ..............
    struct Cache{
        Cache(): cache_is_valid(false){}
        bool cache_is_valid;
        string string_cache;
    };
    mutable Cache *cc;

    RatioCached2(int c=0, int z=1):
        ch(c), zn(z), cc(0) {
            reduce();
    }
    void reduce(){
        //  .........
        if(cc) cc->cache_is_valid = false;
    }
    string toString() const {
        if( !cc )
            cc = new Cache;
        if(cc->cache_is_valid)
            return cc->string_cache;
        else{
            // .... Генеруємо текстове представлення
            cc->cache_is_valid = true;
            return cc->string_cache;
        }
```

У попередньому прикладі вказівник на кеш залишено мутабельним, в ролі оптимізації — він створюється лише коли справді потрібен. Якщо її позбутися, створюючи кеш безпосередньо в конструкторі, код помітно спрощується:

```
class RatioCached2{    ..............
    struct Cache{
        Cache(): cache_is_valid(false){}
        bool cache_is_valid;
        string string_cache;
    };
    Cache *cc; // <======

    RatioCached2(int c=0, int z=1):
        ch(c), zn(z), cc( new Cache ) { // <====
            reduce();
    }
    void reduce(){
        //   .........
        cc->cache_is_valid = false; // <====
    }
    string toString() const {
        if (cc->cache_is_valid)
            return cc->string_cache;
        else{
            // .... Генеруємо текстове представлення
            cc->cache_is_valid = true;
            return cc->string_cache;
        }
    }
};
```

Як і іншими такими засобами, що вимагають підвищеної уваги, користуватися логічно, але не фізично, константними об'єктами, та, зокрема, засобом mutable, слід обережно.

Детальніше про константність, фізичну, логічну і т.д., див. офіційні C++ FAQ: `https://isocpp.org/wiki/faq/const-correctness`, "Const Correctness".